Linux Installation Guide

To install Linux Lite you will need to download the ISO file, then create either a bootable DVD or USB stick from the ISO. To create a bootable DVD, you need to burn the ISO as an "image" to the DVD. Below are listed three free Windows programs that will enable you to do that. If burning to a USB stick, we have also listed a free program that we recommend for doing that in Windows; or a reliable method to create a bootable USB when using a Linux system, or OSX.

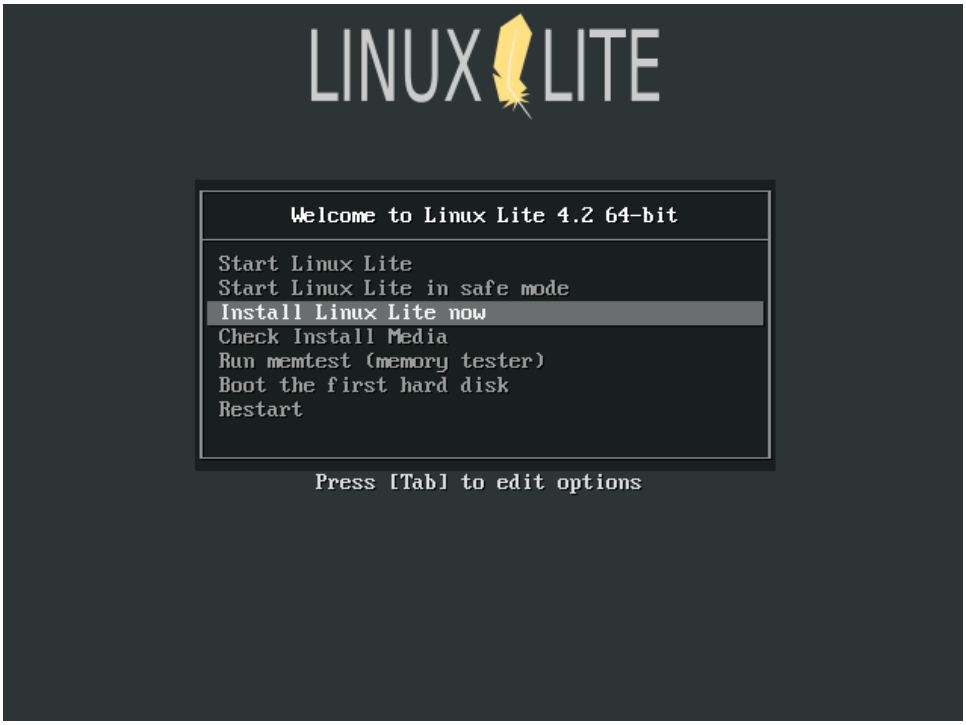
Writing the Linux Lite ISO to USB on Windows

Etcher allows you to create a bootable Live USB drive. Download it from here.

Installing Linux Lite to your computer

Start your computer, and go into your system BIOS and check that the boot order is set so that DVD and USB devices are set to boot first. This varies from BIOS to BIOS. Refer to your motherboard documentation for more information.

Once you are happy with your BIOS set up, insert the DVD disc or USB stick into your computer and reboot your computer. You'll be greeted with the following screen. Let the timer run down if you want to boot to the Live session.



Make sure Start Linux Lite is highlighted, press the enter key to boot the Live session. This is also a good way to check that all of your hardware is supported. If you are having trouble booting and you get just a black screen, try the second option, Start Linux Lite in safe mode.

The other options are:

Install Linux Lite now - boot into Linux Lite live and begin installation straight away.

Check Install Media - run this to check that the Linux Lite image is free from errors.

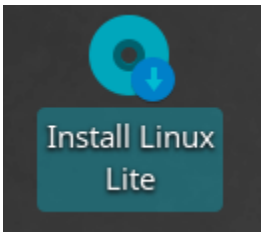
Run memtest (memory tester) - Test your computer memory for faults.

Boot the first hard disk - boot the existing operating system.

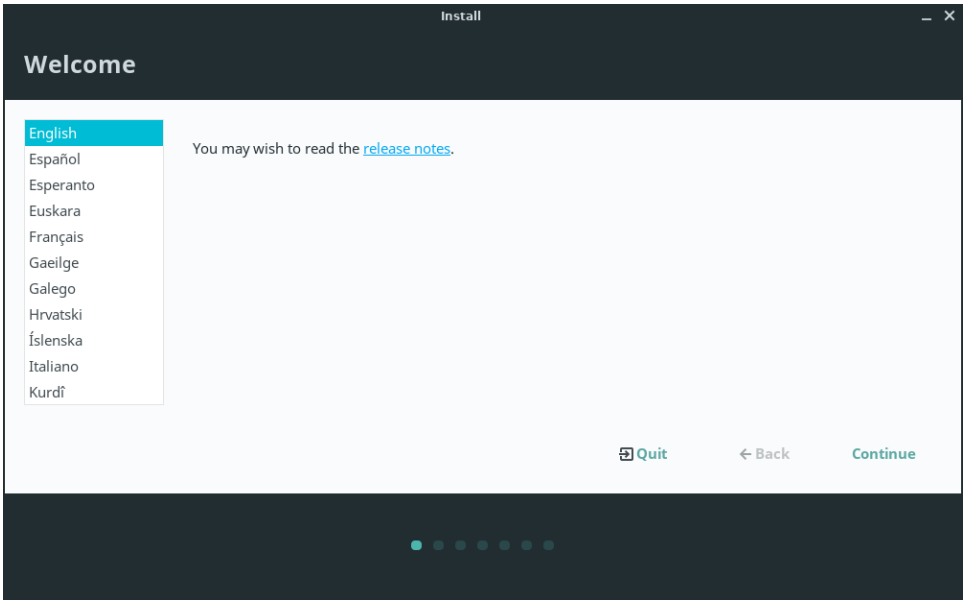
Restart - reboot your computer.

The computer will go through the boot up process, this may take anywhere from a few seconds to a couple of minutes depending on your hardware. The login for the Live user is automatic, there is no need to login. If for some reason you need to login, simply type in linux beneath Other and click on Log In (no password required).

Before we proceed, make sure you have an internet connection first. Once you've had a look around Linux Lite and are ready to install the first step is to double click on the Install Linux Lite icon on your Desktop.



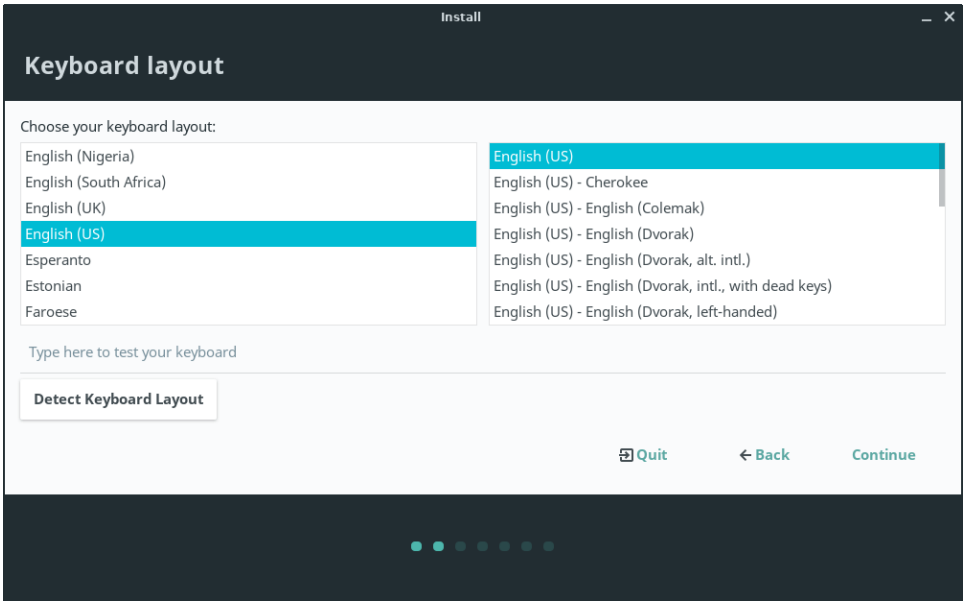
The installer will then open on the desktop and display the first window of the installer. From here you will select your preferred language and view release notes for this version of Linux Lite.



Keyboard layout

The next step is to select your keyboard language and layout. You may select the language that your keyboard corresponds to on the left and any specialized layout on the right. This is the point where you

may select to enable international keys on your keyboard. Though, these options may be changed later. We recommend you leave the default English (US) options here, and adjust your language settings once Linux Lite has been installed and you have rebooted for the first time.

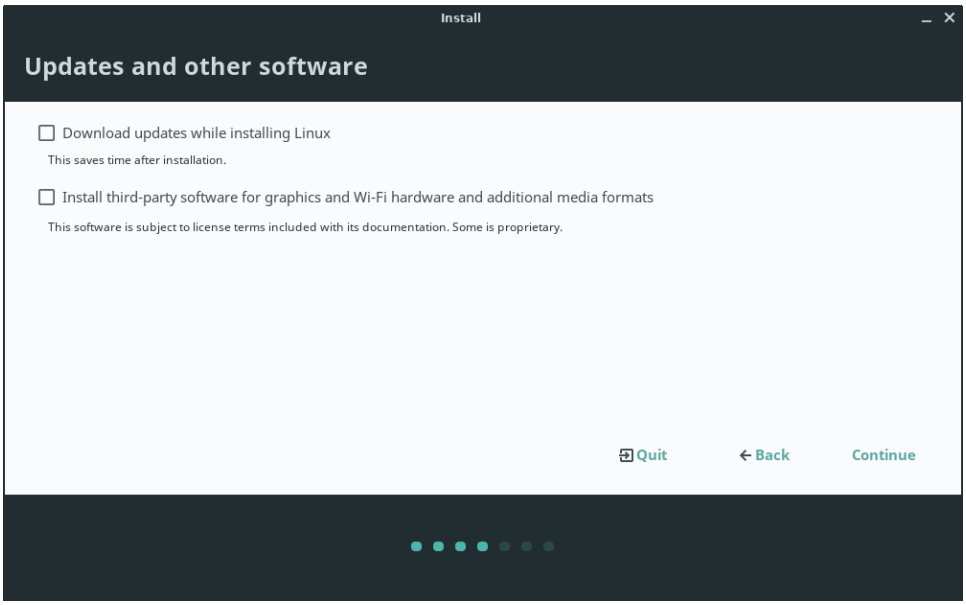


Updates and other software

After selecting continue, you are presented with some requirements for the installation. Ensure that these requirements are met by viewing a check mark beside them. You may also choose to download any available updates while installing or install third-party software during the installation.

NOTE: '*Install third-party software for graphics and Wi-Fi hardware, Flash, MP3 and other media*' Please verify the legal status of the use of the third party software in your country/territory before choosing this option. This option will install codecs for playing media files, and proprietary drivers for some of your hardware. It is a legal requirement for us to inform you of this.

NOTE: '*Download updates while installing Linux*' Generally we don't recommend ticking it as this will significantly slow down the install process. Updating after a fresh install is recommended and will save you time in the long run. A typical install on today's hardware will take around 10-15 minutes. If you check these 2 boxes, the install could take over an hour or longer.



Installation type

Now it is time to choose the method of installing Linux Lite to your hard disk. You will have several options during this phase of the installer.

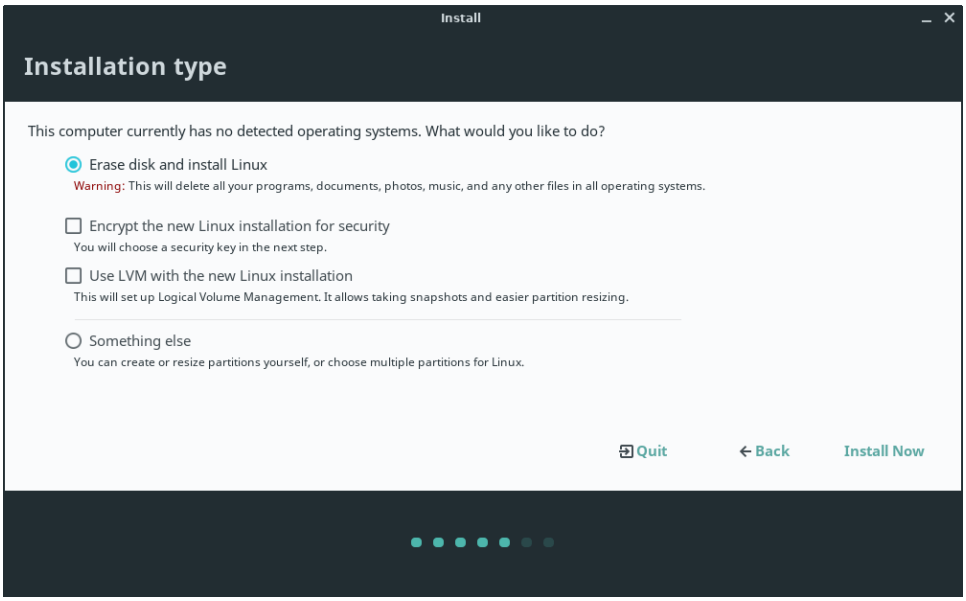
Erase Disk and Install Linux

NOTE: This option will erase your entire hard drive. Make sure this is what you want to do before proceeding.

Upon selecting Erase Disk, the installer will automatically select the default partitioning scheme. Click on Install Now to accept the changes.

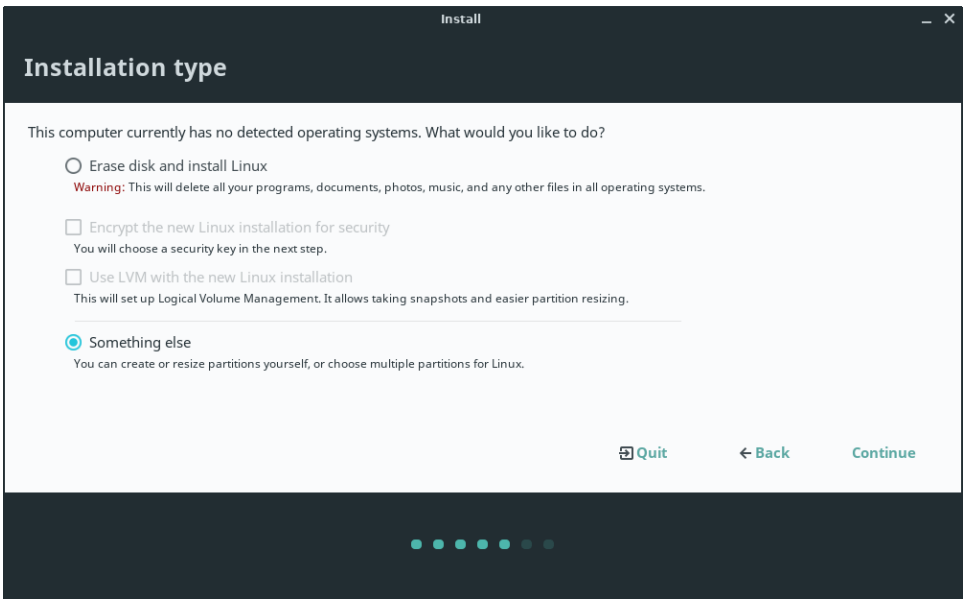
NOTE: This installation option will automatically install the Linux bootloader to the first hard drive on the system. That is typically how most people install and the Linux boot loader offers the choice of booting any other operating systems that are on your computer. However, if your intent is to keep the Windows boot loader in charge of booting the Windows drive and the Windows drive is the first one, then you need to use the 'Something else' installation type. That way you can control the 'Device for boot loader installation' and set it for the specific drive you choose for Linux Lite.

NOTE: If you choose the Encrypt the new Linux installation for security you will be prompted to enter a strong password. This will option will encrypt your entire Linux Lite install. Encrypting is optional and only recommended for people who understand the encryption process.



Something else (dual booting or custom partitioning)

Select Something else from this screen if you wish to dual boot with Windows or edit existing partitions (or both).



Keeping Windows (dual booting)

In this example, we have 3 partitions.

sda1 - The recovery partition for Windows.

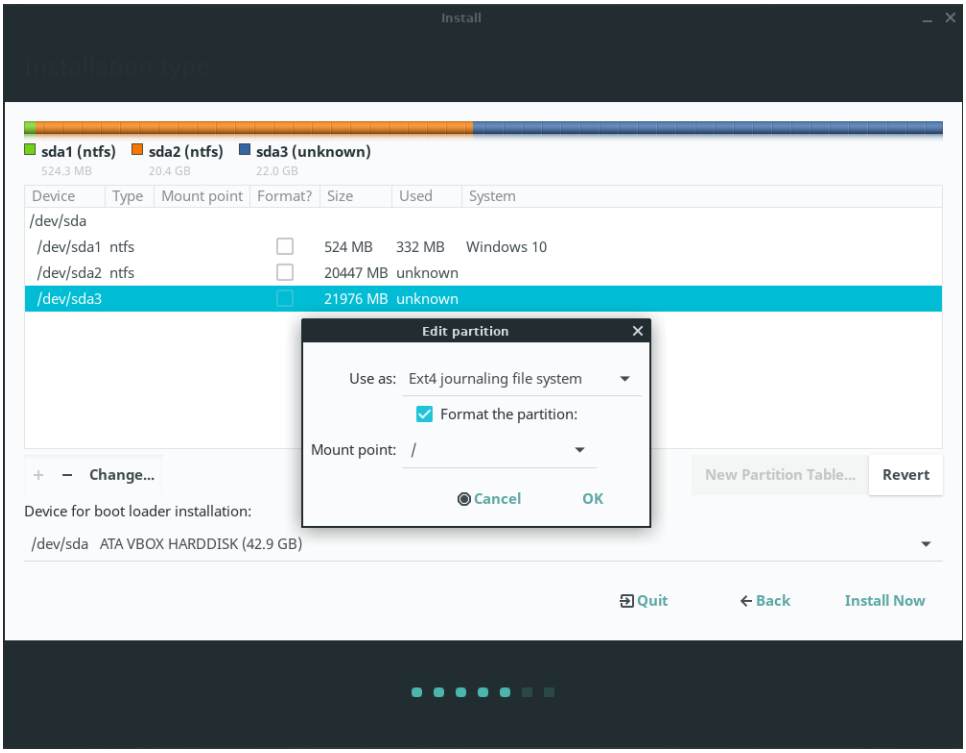
sda2 - Where Windows is installed (note ntfs is the Type).

sda3 - The spare, unformatted partition. This is where we will install Linux Lite to.

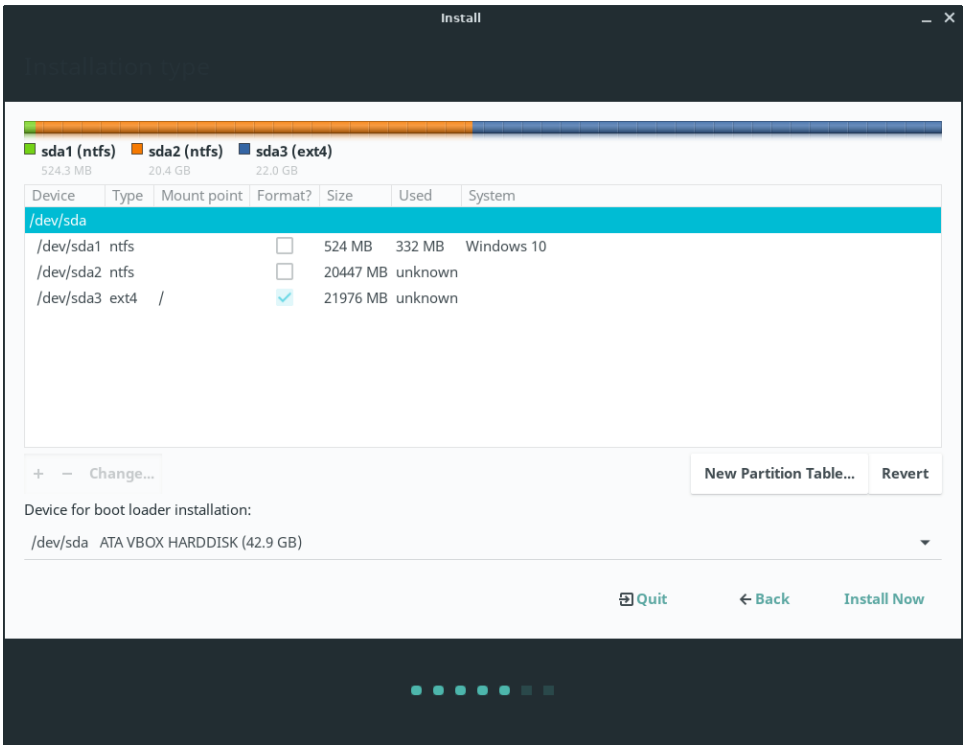
NOTE: If you have any other NTFS drives, it is important to give these a mount point during the install. That way they will be easily accessible after the installation.

The mount point is simply "where" in the file system you want that partition's files to show up. You can name the mount point what ever you want, be sure to give it a simple, no spaces lowercase name like eg. /myntfs do not use /My NTFS. **DO NOT select 'Format the partition' when editing any NTFS partitions.**

Click on the sda3 partition and click the Change button. Select Ext4 journaling file system, place a tick in Format the partition and select / as the Mount point. Click OK.

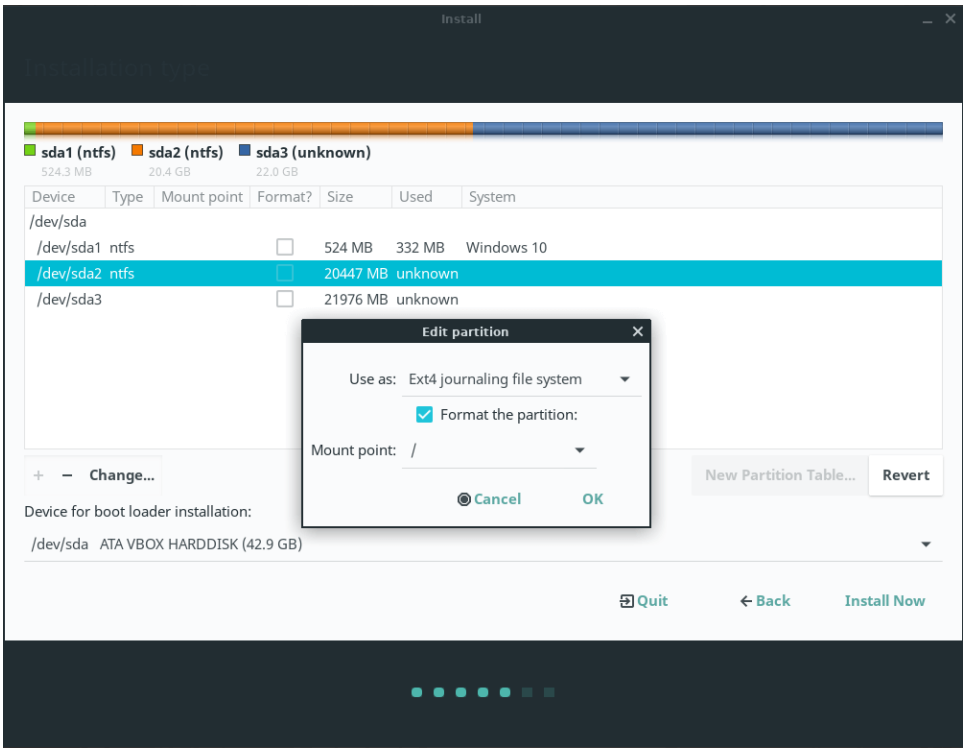


When you have finished, click on Install Now.



Over writing Windows

In this example, we select the sda2 partition (where Windows is installed) and click the Change button. Select Ext4 journaling file system, place a tick in Format the partition and select / as the Mount point. Click OK.



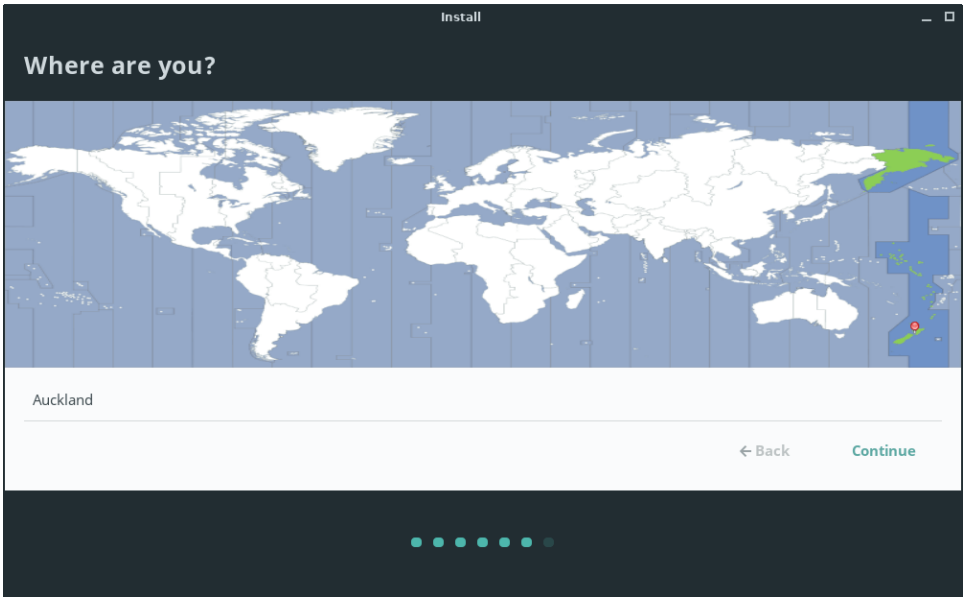
NOTE: Device for boot loader installation.

The Linux boot loader offers the choice of booting any other operating systems that are on your computer. In this example since we are over writing Windows, we simply select sda for the boot loader.

Whichever method you choose, once you are finished, click on Install Now.

Where are you?

After completing the partitioning phase of the installation, you are prompted to set up some personal details about the system. The installer will continue to work in the background while you are filling out this information. The first step is to select your time zone. The installer may detect your time zone based on your location via your ISP's information. If not, you may select your time zone by either clicking on your region on the supplied map or by typing in your area in the box below.



Who are you?

On the next screen, you are greeted by several fields to fill. These fields include:

Your name - your real, full name.

Your computer's name - the name that you wish to use to identify the computer on your network.

Pick a username - your user name that is used to login to the system.

Choose a password - your password that you wish to use for your user and sudo (administrator) password.

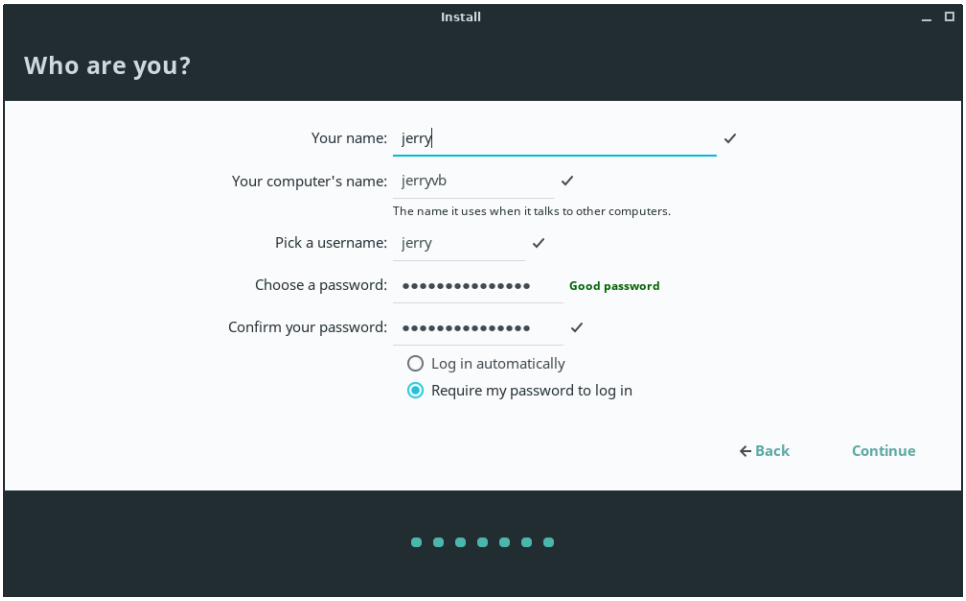
Confirm your password - enter the above password again.

NOTE: Choose a strong password, a mix of upper and lower case as well as a mix of numbers and characters. An example of this would be: **y%9L2x#o** would be considered a strong password. Use a program like KeePassXC to keep all your passwords safely stored and NEVER use the same password for

different websites. (KeePassXC is available for installation from Menu, Settings, Lite Software. You do not need to download it from their website.)

NEVER COMPROMISE YOUR SECURITY FOR CONVENIENCE.

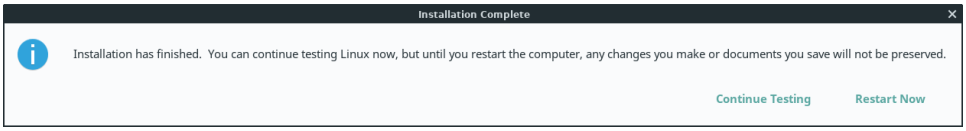
There are also options to enable auto-login for your user or have the computer prompt you for a password everytime you login.



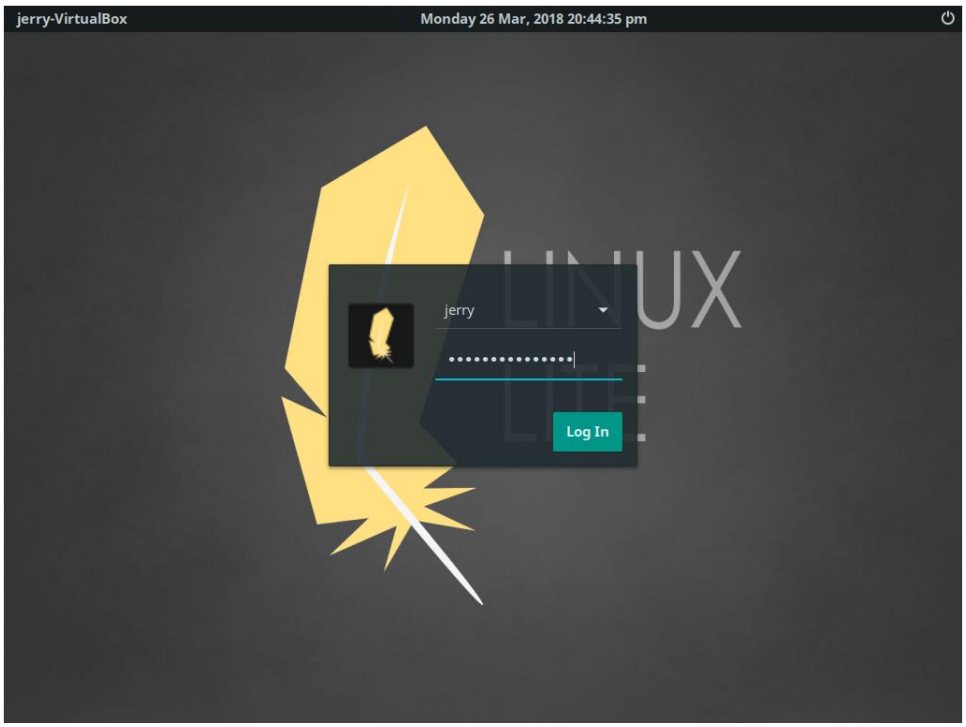
If you have a webcam, you may encounter a screen during the install asking to Take a photo this feature is supported so that when you first log in, your photo will be displayed as your avatar. If you skip this step, you can use a .png photo of yourself, rename it to .face and place it in your **/home/user** folder. By default, the Linux Lite logo will represent your login icon.

After finishing all of these steps you will be greeted with a slideshow detailing all of the features and software that are included in this version of Linux Lite. You may allow them to play on their own or navigate through them using the arrow buttons on the sides of the window (the picture below may vary from version to version).

Once the installation is complete you will be prompted to either restart your computer and begin using your newly installed Linux Lite system or to continue using the testing environment on the DVD/USB that you are using. If for some reason your computer hangs on restart, press F4 or Enter.



After a reboot you will be greeted by the login screen:



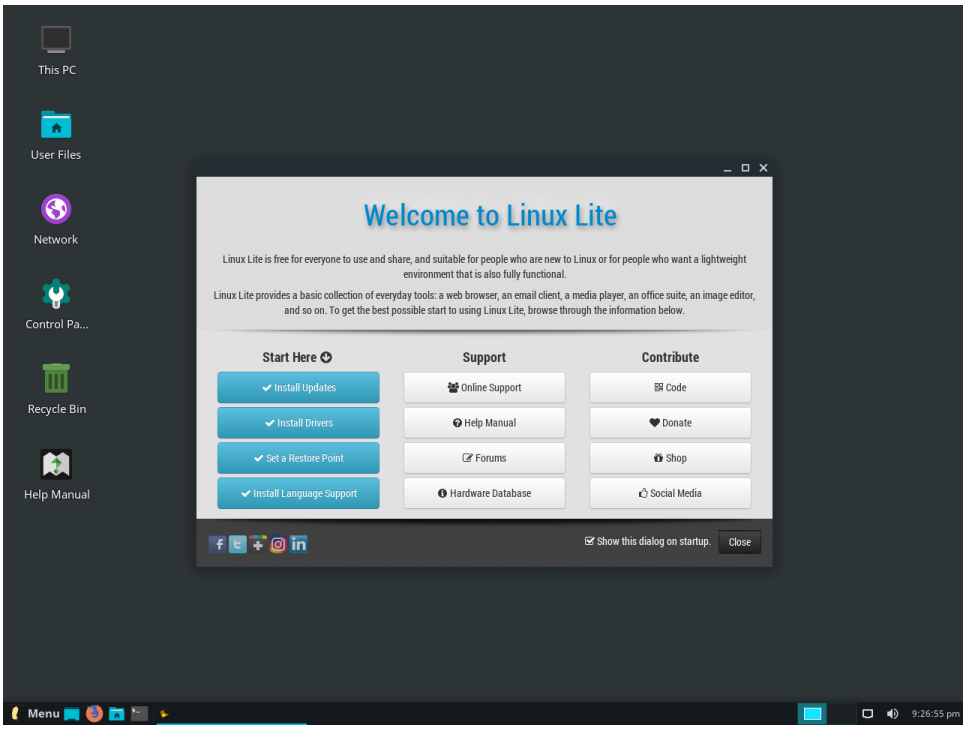
After you have logged in you will see the desktop and Welcome window. Be sure to read through the Welcome window to get you off to the best possible start to using Linux Lite.

Install Updates - do this first (reboot if prompted to after updates have finished).

Install Drivers - see if there are any drivers needed for your computer.

Set a Restore Point - set your first restore point using Timeshift, similar to System Restore in Windows.

Install Language Support - if you are using anything other than English.



Linux Device Driver Tutorial for Beginners

Linux operating system contains 3 main sections: Root File System, Kernel and Bootloader.

Root File System:

This part of the OS contains application binaries, libraries, scripts, config files and kernel loadable module files etc..

Kernel:

This part is the heart of OS, the Kernel is responsible for handling all the operations needed to run the OS such as memory management, process management, and input/output hardware operations etc..

Bootloader:

This is the first part to be executed by the CPU on boot. Bootloader contains the source code to initialize the system and start execute the kernel and contains commands for debugging and modifying the kernel environment, it also contains the commands to download and update the kernel and system images into the flash memory.

Drivers act as a bridge between hardware and a user application, the Kernel provides a mechanism called system calls to talk to the kernel. In Linux, drivers can be implemented in two ways, one is drivers can be compiled as part of the kernel and another one is drivers can be compiled as modules and loaded at runtime.

Let us start with a simple hello world kernel module. Here is the source code for a simple hello world kernel module.

hello.c

```
#include<linux/module.h>  //needed for module_init and module_exit
#include<linux/kernel.h>  //needed for KERN_INFO
#include<linux/init.h>        //needed for the macros

int __init hw_init(void) {
        printk(KERN_INFO"Hello World\n");
        return 0;
}

void __exit hw_exit(void) {
        printk(KERN_INFO"Bye World\n");
}

MODULE_LICENSE("GPL");

module_init(hw_init);
module_exit(hw_exit);
```

Makefile

```
obj-m := hello.o
all:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
clean:
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

Create a folder named **hello** and then place the **hello.c** and **Makefile** inside it. Open the **terminal** application and change directory to hello. Now run the command **make** and if it is successful then it should generate loadable kernel module file called **hello.ko**.

When you run make if you get output **make: Nothing to be done for `all'.** Then please make sure that in the Makefile you have entered tab(no spaces) before make -C. If make is successful you should get the output as shown below.

```
make[1]: Entering directory `/usr/src/linux-headers-3.13.0-128-generic'  CC
[M] /home/tejas/Desktop/hello/hello.o  Building modules, stage 2.  MODPOST 1
modules  CC /home/tejas/Desktop/hello/hello.mod.o  LD [M]
/home/tejas/Desktop/mvs/pers/kern/hello/hello.komake[1]: Leaving directory
`/usr/src/linux-headers-3.13.0-128-generic'
```

Now let us test the module by loading it into the kernel. For loading and unloading kernel modules we need to have superuser permission. Use the following command to load the kernel module into the kernel.

```
sudo insmod hello.ko
```

To see the printk message you need to check the kernel log, to check the kernel log use the following command.

dmesg

This command will output the kernel log messages, at the end you should see that our message **Hello World** printed.

To unload the module use the following command.

```
sudo rmmod hello
```

To see the printk message use dmesg command again and in the kernel log you can see our message **Bye World**.

Now let us understand the source code.

hello.c

To start writing the kernel driver you can use any editor or ide of your choice but most commonly kernel developers prefer using **vi** editor.

Every kernel module should include the header file **linux/module.h** this has the declarations and macros for kernel functions such as **module_init** and **module_exit** etc. The two most necessary functions for a kernel driver are module_init and module_exit functions. The function whose pointer is passed to module_init will be executed when we load the module into the kernel, and the function whose pointer is passed to module_exit will be called when we unload or remove the module from the kernel.

Inside the kernel for debugging and printing the log, we use **printk** function which is similar to printf function which we use in the application. You can use the macros such as KERN_INFO, KERN_ERR etc.. to specify a log level.

If we are writing a driver to talk to a specific hardware then the init function should have the code to initialize the hardware before we start using it and exit function should have a code to clean up resources(Dynamic Memory etc) we used in the driver before we exit from the kernel.

Here in this example, we are just printing debug messages in init and exit functions.

Makefile

For building the kernel module we need to write a Makefile which will guide **make** utility how to compile the module. The syntax **obj-m** is used to tell the kernel makefile that the driver needs to be compiled as module using the specified object file. When you just run command **make** then the control comes to the **all:** section of the Makefile and if you run command **make clean** then the control goes to the **clean:** section of Makefile. From this Makefile we are actually running make inside the kernel source directory using option -C. Please make sure you have kernel source directory installed in your system. Here in this example we used command **uname -r** to find the current version of your system's linux kernel.

We have used option M=$(PWD) to indicate in the kernel makefile that the source for driver is in present working directory and we are specifying the word **modules** to tell kernel makefile to just build

modules and not to build the complete kernel source code. In **clean:** section of Makefile we are telling kernel makefile to clean the object files generated to build this module.

This should you get started compiling and running your first kernel module.

Files in Linux

From top to bottom, the directories you are seeing are as follows.

/bin

/bin is the directory that contains *bin*aries, that is, some of the applications and programs you can run. You will find the *ls* program mentioned above in this directory, as well as other basic tools for making and removing files and directories, moving them around, and so on. There are more *bin* directories in other parts of the file system tree, but we'll be talking about those in a minute.

/boot

The */boot* directory contains files required for starting your system. Do I have to say this? Okay, I'll say it: **DO NOT TOUCH!**. If you mess up one of the files in here, you may not be able to run your Linux and it is a pain to repair. On the other hand, don't worry too much about destroying your system by accident: you have to have superuser privileges to do that.

/dev

/dev contains *dev*ice files. Many of these are generated at boot time or even on the fly. For example, if you plug in a new webcam or a USB pendrive into your machine, a new device entry will automagically pop up here.

/etc

/etc is the directory where names start to get confusing. */etc* gets its name from the earliest Unixes and it was literally "et cetera" because it was the dumping ground for system files administrators were not sure where else to put.

Nowadays, it would be more appropriate to say that *etc* stands for "Everything to configure," as it contains most, if not all system-wide configuration files. For example, the files that contain the name of your system, the users and their passwords, the names of machines on your network and when and where the partitions on your hard disks should be mounted are all in here. Again, if you are new to Linux, it may be best if you don't touch too much in here until you have a better understanding of how things work.

/home

/home is where you will find your users' personal directories. In my case, under */home* there are two directories: */home/paul*, which contains all my stuff; and */home/guest*, in case anybody needs to borrow my computer.

/lib

/lib is where *lib*raries live. Libraries are files containing code that your applications can use. They contain snippets of code that applications use to draw windows on your desktop, control peripherals, or send files to your hard disk.

There are more *lib* directories scattered around the file system, but this one, the one hanging directly off of */* is special in that, among other things, it contains the all-important kernel modules. The kernel modules are drivers that make things like your video card, sound card, WiFi, printer, and so on, work.

/media

The */media* directory is where external storage will be automatically mounted when you plug it in and try to access it. As opposed to most of the other items on this list, */media* does not hail back to 1970s, mainly because inserting and detecting storage (pendrives, USB hard disks, SD cards, external SSDs, etc) on the fly, while a computer is running, is a relatively new thing.

/mnt

The */mnt* directory, however, is a bit of remnant from days gone by. This is where you would manually mount storage devices or partitions. It is not used very often nowadays.

/opt

The */opt* directory is often where software you compile (that is, you build yourself from source code and do not install from your distribution repositories) sometimes lands. Applications will end up in the */opt/bin* directory and libraries in the */opt/lib* directory.

A slight digression: another place where applications and libraries end up in is */usr/local*, When software gets installed here, there will also be */usr/local/bin* and */usr/local/lib* directories. What determines which software goes where is how the developers have configured the files that control the compilation and installation process.

/proc

/proc, like */dev* is a virtual directory. It contains information about your computer, such as information about your CPU and the kernel your Linux system is running. As with */dev*, the files and directories are generated when your computer starts, or on the fly, as your system is running and things change.

/root

/root is the home directory of the superuser (also known as the "Administrator") of the system. It is separate from the rest of the users' home directories BECAUSE YOU ARE NOT MEANT TO TOUCH IT. Keep your own stuff in you own directories, people.

/run

/run is another new directory. System processes use it to store temporary data for their own nefarious reasons. This is another one of those DO NOT TOUCH folders.

/sbin

/sbin is similar to */bin*, but it contains applications that only the superuser (hence the initial *s*) will need. You can use these applications with the `sudo` command that temporarily concedes you superuser powers on many distributions. */sbin* typically contains tools that can install stuff, delete stuff and format

stuff. As you can imagine, some of these instructions are lethal if you use them improperly, so handle with care.

/usr

The */usr* directory was where users' home directories were originally kept back in the early days of UNIX. However, now */home* is where users kept their stuff as we saw above. These days, */usr* contains a mish-mash of directories which in turn contain applications, libraries, documentation, wallpapers, icons and a long list of other stuff that need to be shared by applications and services.

You will also find *bin*, *sbin* and *lib* directories in */usr*. What is the difference with their root-hanging cousins? Not much nowadays. Originally, the */bin* directory (hanging off of root) would contain very basic commands, like `ls`, `mv` and `rm`; the kind of commands that would come pre-installed in all UNIX/Linux installations, the bare minimum to run and maintain a system. */usr/bin* on the other hand would contain stuff the users would install and run to use the system as a work station, things like word processors, web browsers, and other apps.

But many modern Linux distributions just put everything into */usr/bin* and have */bin* point to */usr/bin* just in case erasing it completely would break something. So, while Debian, Ubuntu and Mint still keep */bin* and */usr/bin* (and */sbin* and */usr/sbin*) separate; others, like Arch and its derivatives just have one "real" directory for binaries, */usr/bin*, and the rest or **bin*s are "fake" directories that point to */usr/bin*.

/srv

The */srv* directory contains data for servers. If you are running a web server from your Linux box, your HTML files for your sites would go into */srv/http* (or */srv/www*). If you were running an FTP server, your files would go into */srv/ftp*.

/sys

/sys is another virtual directory like */proc* and */dev* and also contains information from devices connected to your computer.

/tmp

/tmp contains temporary files, usually placed there by applications that you are running. The files and directories often (not always) contain data that an application doesn't need right now, but may need later on.

You can also use */tmp* to store your own temporary files -- */tmp* is one of the few directories hanging off / that you can actually interact with without becoming superuser.

/var

/var was originally given its name because its contents was deemed *variable*, in that it changed frequently. Today it is a bit of a misnomer because there are many other directories that also contain data that changes frequently, especially the virtual directories we saw above.

Be that as it may, */var* contains things like logs in the */var/log* subdirectories. Logs are files that register events that happen on the system. If something fails in the kernel, it will be logged in a file in */var/log*; if someone tries to break into your computer from outside, your firewall will also log the attempt here. It also contains *spools* for tasks. These "tasks" can be the jobs you send to a shared printer when you have

to wait because another user is printing a long document, or mail that is waiting to be delivered to users on the system.

Your system may have some more directories we haven't mentioned above. In the screenshot, for example, there is a */snap* directory. That's because the shot was captured on an Ubuntu system. Ubuntu has recently incorporated snap packages as a way of distributing software. The */snap* directory contains all the files and the software installed from snaps.

Linux command to check disk space using:

1. `df` command – Shows the amount of disk space used and available on Linux file systems.
2. `du` command – Display the amount of disk space used by the specified files and for each subdirectory.

Linux check disk space with df command

1. Open the terminal and type the following command to check disk space.
2. The basic syntax for df is:

```
df [options] [devices]
```

3. Type:

```
df
```

Sample outputs:

```
root@nas02:~# df
Filesystem      1K-blocks     Used  Available Use% Mounted on
udev                10240        0      10240   0% /dev
tmpfs              610488     9700     600788   2% /run
/dev/sdc1        14932444  7835788    6331472  56% /
tmpfs             1526216        0    1526216   0% /dev/shm
tmpfs                5120        0       5120   0% /run/lock
tmpfs             1526216        0    1526216   0% /sys/fs/cgroup
/dev/sda       2930266584 69405248 2859579472   3% /data
tmpfs              305244        0     305244   0% /run/user/0
root@nas02:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev             10M     0   10M   0% /dev
tmpfs           597M  9.5M  587M   2% /run
/dev/sdc1        15G  7.5G  6.1G  56% /
tmpfs           1.5G     0  1.5G   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
tmpfs           1.5G     0  1.5G   0% /sys/fs/cgroup
/dev/sda        2.8T   67G  2.7T   3% /data
tmpfs           299M     0  299M   0% /run/user/0
root@nas02:~#
```

How to Create a New User in Linux

To create a new user account named `username` using the `useradd` command you would run:

```
useradd username
```

In it's simplest form when used without any option, useradd will create a new user account with the default settings specified in the `/etc/default/useradd` file.

The command adds an entry to the `/etc/passwd`, `/etc/shadow`, `/etc/group` and `/etc/gshadow` files.

To be able to login as the newly created user, you need to set the user password. To do that run the `passwd` command followed by the username:

```
passwd username
```

You will be prompted to enter and confirm the password. Make sure you use a strong password.

```
Changing password for user username.New password:Retype new password:passwd:
all authentication tokens updated successfully.
```

Modifying a User with usermod

In the below example we will add the user to the "folks" group and change his shell to ksh; To do this we will use the usermod command.

```
usermod -aG folks -s /bin/ksh someuser
```

The usermod command utilizes the same flags as useradd, however in the example above there is a new flag `-a`. The `-a` flag is given to tell usermod to add the user to the folks group. Without the `-a` flag the user someuser would have been removed from the specialpeople and others group and added to the folks group.

Deleting Users

```
userdel -r someuser
```

The `-r` flag in the above command tells userdel to also remove the user's home directory, this is not the default action and is good practice if you are truly deleting the user for good and do not want the contents of their home directory.

Important Linux Commands

File Commands

File Administration

ls `[option(s)] [file(s)]`

If you run **ls** without any additional parameters, the program will list the contents of the current directory in short form.

-l

detailed list

-a

displays hidden files

cp `[option(s)] sourcefile targetfile`

Copies `sourcefile` to `targetfile`.

-i

Waits for confirmation, if necessary, before an existing `targetfile` is overwritten

-r

Copies recursively (includes subdirectories)

mv `[option(s)] sourcefile targetfile`

Copies `sourcefile` to `targetfile` then deletes the original `sourcefile`.

-b

Creates a backup copy of the `sourcefile` before moving

-i

Waits for confirmation, if necessary, before an existing targetfile is overwritten

rm `[option(s)] file(s)`

Removes the specified files from the file system. Directories are not removed by **rm** unless the option `-r` is used.

-r

Deletes any existing subdirectories

-i

Waits for confirmation before deleting each file.

ln `[option(s)] sourcefile targetfile`

Creates an internal ***link*** *from the* `sourcefile` to the `targetfile`, under a different name. Normally, such a link points directly to the `sourcefile` on one and the same file system. However, if **ln** is executed with the `-s` option, it creates a symbolic link that only points to the directory where the `sourcefile` is located, thus enabling linking across file systems.

-s

Creates a symbolic link

cd `[options(s)] [directory]`

Changes the current directory. **cd** without any parameters changes to the user's home directory.

mkdir `[option(s)] directoryname`

Creates a new directory.

rmdir `[option(s)] directoryname`

Deletes the specified directory, provided it is already empty.

chown `[option(s)] username.group file(s)`

Transfers the ownership of a file to the user with the specified user name.

-R

Changes files and directories in all subdirectories.

chgrp `[option(s)] groupname file(s)`

Transfers the group ownership of a given `file` to the group with the specified group name. The file owner can only change group ownership if a member of both the existing and the new group.

chmod `[options] mode file(s)`

Changes the access permissions.

The `mode` parameter has three parts: `group`, `access`, and `access type`. `group` accepts the following characters:

u

user

g

group

o

others

For `access`, access is granted by the + symbol and denied by the `-` symbol.

The `access type` is controlled by the following options:

r

read

w

write

x

eXecute — executing files or changing to the directory.

s

Set uid bit — the application or program is started as if it were started by the owner of the file.

gzip `[parameters] file(s)`

This program compresses the contents of files, using complex mathematical algorithms. Files compressed in this way are given the extension `.gz` and need to be uncompressed before they can be used. To compress several files or even entire directories, use the **tar** command.

-d

decompresses the packed gzip files so they return to their original size and can be processed normally (like the command **gunzip**).

tar `options archive file(s)`

The **tar** puts one file or (usually) several files into an archive. Compression is optional.

tar is a quite complex command with a number of options available. The most frequently used options are:

-f

Writes the output to a file and not to the screen as is usually the case

-c

Creates a new tar archive

-r

Adds files to an existing archive

-t

Outputs the contents of an archive

-u

Adds files, but only if they are newer than the files already contained in the archive

-x

Unpacks files from an archive (*extraction*)

-z

Packs the resulting archive with **gzip**

-j

Compresses the resulting archive with **bzip2**

-v

Lists files processed

The archive files created by **tar** end with `.tar`. If the tar archive was also compressed using **gzip**, the ending is `.tgz` or `.tar.gz`. If it was compressed using **bzip2**, `.tar.bz2`.

locate `pattern(s)`

The locate command can find in which directory a specified file is located. If desired, use ***wild card*** *to specify file names. The program is very speedy, as it uses a database specifically created for the purpose (rather than searching through the entire file system). This very fact, however, also results in a major drawback: locate is unable to find any files created after the latest update of its database.*

The database can be generated by `root` with **updatedb**.

updatedb `[options(s)]`

This command performs an update of the database used by **locate**. To include files in all existing directories, run the program as `root`. It also makes sense to place it in the background by appending an ampersand (&), so you can immediately continue working on the same command line (**updatedb &**).

find `[option(s)]`

The **find** command allows you to search for a file in a given directory. The first argument specifies the directory in which to start the search. The option **-name** must be followed by a search string, which may also include ***wild cards***. *Unlike* **locate**, which uses a database, **find** scans the actual directory.

Commands to Access File Contents

cat `[option(s)] file(s)`

The **cat** command displays the contents of a file, printing the entire contents to the screen without interruption.

-n

Numbers the output on the left margin

less `[option(s)] file(s)`

This command can be used to browse the contents of the specified file. Scroll half a screen page up or down with **PgUp** and **PgDn** or a full screen page down with **Space**. Jump to the beginning or end of a file using **Home** and **End**. Press **Q** to exit the program.

grep `[option(s)] searchstring filenames`

The grep command finds a specific `searchstring` in the specified `file(s)`. If the search string is found, the command displays the line in which the `searchstring` was found along with the file name.

-i

Ignores case

-l

Only displays the names of the respective files, but not the text lines

-n

Additionally displays the numbers of the lines in which it found a hit

-l

Only lists the files in which searchstring does not occur

diff `[option(s)] file1 file2`

The **diff** command compares the contents of any two files. The output produced by the program lists all lines that do not match.

This is frequently used by programmers who need only send their program alterations and not the entire source code.

-q

Only reports *whether* the two given files differ

File Systems

mount `[option(s)] [<device>] mountpoint`

This command can be used to mount any data media, such as hard disks, CD-ROM drives, and other drives, to a directory of the Linux file system.

-r

mount read-only

-t filesystem

Specifies the file system. The most common are `ext2` for Linux hard disks, `msdos` for MS-DOS media, `vfat` for the Windows file system, and `iso9660` for CDs.

For hard disks not defined in the file `/etc/fstab`, the device type must also be specified. In this case, only `root` can mount. If the file system should also be mounted by other users, enter the option `user` in the appropriate line in the `/etc/fstab` file (separated by commas) and save this change. Further information is available in mount.

umount `[option(s)] mountpoint`

This command unmounts a mounted drive from the file system. To prevent data loss, run this command before taking a removable data medium from its drive. Normally, only `root` is allowed to run the commands **mount** and **umount**. To enable other users to run these commands, edit the `/etc/fstab` file to specify the option `user` for the respective drive.

System Commands

System Information

df `[option(s)] [directory]`

The **df** (disk free) command, when used without any options, displays information about the total disk space, the disk space currently in use, and the free space on all the mounted drives. If a directory is specified, the information is limited to the drive on which that directory is located.

-H

shows the number of occupied blocks in gigabytes, megabytes, or kilobytes — in human-readable format

-t

Type of file system (ext2, nfs, etc.)

du `[option(s)] [path]`

This command, when executed without any parameters, shows the total disk space occupied by files and subdirectories in the current directory.

-a

Displays the size of each individual file

-h

Output in human-readable form

-s

Displays only the calculated total size

free `[option(s)]`

The command **free** displays information about RAM and swap space usage, showing the total and the used amount in both categories.

-b

Output in bytes

-k

Output in kilobytes

-m

Output in megabytes

date `[option(s)]`

This simple program displays the current system time. If run as `root`, it can also be used to change the system time. Details about the program are available in date.

Processes

top `[options(s)]`

top provides a quick overview of the currently running ***processes****. Press* **H** to access a page that briefly explains the main options to customize the program.

ps `[option(s)] [process ID]`

If run without any options, this command displays a table of all *your own* programs or processes — those you started. The options for this command are not preceded by hyphen.

aux

Displays a detailed list of all processes, independent of the owner.

kill `[option(s)] process ID`

Unfortunately, sometimes a program cannot be terminated in the normal way. However, in most cases, you should still be able to stop such a runaway program by executing the **kill** command, specifying the respective process ID (see **top** and **ps**).

kill sends a *TERM* signal that instructs the program to shut itself down. If this does not help, the following parameter can be used:

-9

Sends a *KILL* signal instead of a *TERM* signal, with which the process really is *annihilated* by the operating system. This brings the specific processes to an end in almost all cases.

killall `[option(s)] processname`

This command is similar to **kill**, but uses the process name (instead of the process ID) as an argument, causing all processes with that name to be killed.

Network

ping `[option(s)] host name|IP address`

The ping command is the standard tool for testing the basic functionality of TCP/IP networks. It sends a small data packet to the destination host, requesting an immediate reply. If this works, ping displays a message to that effect, which indicates that the network link is basically functioning.

-c

`number` Determines the total number of packages to send and ends after they have been dispatched. By default, there is no limitation set.

-f

flood ping: sends as many data packages as possible. A popular means, reserved to `root`, to test networks.

-i

`value` Specifies the interval between two data packages in seconds. Default: one second

nslookup

The Domain Name System resolves domain names to IP addresses. With this tool, send queries to information servers (DNS servers).

telnet `[option(s)] host name or IP address`

Telnet is actually an Internet protocol that enables you to work on remote hosts across a network. telnet is also the name of a Linux program that uses this protocol to enable operations on remote computers.

Warning

Do not use telnet over a network on which third parties can *eavesdrop.* Particularly on the Internet, use encrypted transfer methods, such as **ssh**, to avoid the risk of malicious misuse of a password (see the man page for **ssh**).

Miscellaneous

passwd `[option(s)] [username]`

Users may change their own passwords at any time using this command. Furthermore, the administrator `root` can use the command to change the password of any user on the system.

su `[option(s)] [username]`

The **su** command makes it possible to log in under a different user name from a running session. When using the command without specifying a user name, you will be prompted for the `root` password. Specify a user name and the corresponding password to use the environment of the respective user. The password is not required from ***root***, *as* `root` is authorized to assume the identity of any user.

halt `[option(s)]`

To avoid loss of data, you should always use this program to shut down your system.

reboot `[option(s)]`

Does the same as **halt** with the difference that the system performs an immediate reboot.

clear

This command cleans up the visible area of the console. It has no options.